

ARQUITECTURA DEL SOFTWARE

Sistema Bancario EurekaBank

Servicios Web SOAP con Java y MySQL en Railway

Versión 1.0

Diciembre 2024

1. INTRODUCCIÓN	3
1.1 Propósito	3
1.2 Alcance	3
2. VISIÓN ARQUITECTÓNICA	4
2.1 Estilo Arquitectónico	4
2.2 Arquitectura de Tres Capas	4
3. COMPONENTES DEL SERVIDOR.....	5
3.1 Servicios Web SOAP	5
3.1.1 ServicioAutenticacion	5
3.1.2 ServicioTransaccion	5
3.1.3 ServicioReporte.....	5
3.2 Capa de Servicios.....	5
3.3 Capa de Acceso a Datos (DAO)	6
3.4 Objetos de Transferencia (DTO).....	6
4. PATRONES DE DISEÑO	7
4.1 Patrón DAO (Data Access Object).....	7
4.2 Patrón DTO (Data Transfer Object)	7
4.3 Patrón Service Layer	7
4.4 Patrón Singleton (Gestor de Conexiones)	7
5. INFRAESTRUCTURA Y DESPLIEGUE.....	8
5.1 Base de Datos en Railway	8
5.2 Servidor de Aplicaciones	8
5.3 Requisitos del Sistema	8
6. SEGURIDAD	9
6.1 Autenticación	9

1. INTRODUCCIÓN

Este documento describe la arquitectura del Sistema Bancario EurekaBank, detallando su estructura organizacional, componentes principales, patrones de diseño utilizados y decisiones arquitectónicas clave. El sistema implementa una arquitectura orientada a servicios (SOA) basada en servicios web SOAP, que permite la interoperabilidad entre diferentes plataformas y tecnologías.

1.1 Propósito

El propósito de este documento es proporcionar una visión completa de la arquitectura del sistema, facilitando la comprensión del diseño por parte del equipo de desarrollo, arquitectos, y stakeholders técnicos. Además, sirve como referencia para futuras extensiones y mantenimiento del sistema.

1.2 Alcance

Este documento cubre:

- Arquitectura general del sistema a tres capas
- Componentes del servidor SOAP en Java
- Clientes multiplataforma en .NET
- Base de datos MySQL en Railway
- Patrones de diseño y mejores prácticas aplicadas

2. VISIÓN ARQUITECTÓNICA

2.1 Estilo Arquitectónico

El sistema EurekaBank está diseñado siguiendo el estilo arquitectónico de Arquitectura Orientada a Servicios (SOA) combinado con una arquitectura en capas de tres niveles. Esta combinación proporciona:

- **Separación de responsabilidades:** Cada capa tiene responsabilidades bien definidas
- **Reutilización:** Los servicios web pueden ser consumidos por múltiples clientes
- **Escalabilidad:** Cada capa puede escalarse independientemente
- **Mantenibilidad:** Los cambios en una capa no afectan a las demás
- **Interoperabilidad:** SOAP permite comunicación entre diferentes tecnologías

2.2 Arquitectura de Tres Capas

Capa	Descripción y Componentes
Capa de Presentación	Tecnología: .NET Framework / .NET 6+ Componentes: • Cliente Consola (aplicación de línea de comandos) • Cliente Escritorio (Windows Forms) • Cliente Web (ASP.NET) • Cliente Móvil (Xamarin/MAUI) Responsabilidad: Interfaz de usuario, captura de datos, presentación de resultados
Capa de Lógica de Negocio	Tecnología: Java con Jakarta EE, SOAP Componentes: • Servicios Web SOAP (3 servicios principales) • Capa de Servicios (Business Logic) • Objetos de Transferencia de Datos (DTOs) • Validadores y Utilidades Responsabilidad: Procesamiento de transacciones, validaciones, reglas de negocio, cálculos
Capa de Datos	Tecnología: MySQL 8.0 en Railway Componentes: • Objetos de Acceso a Datos (DAOs) • Gestor de Conexiones (ConexionDB) • Base de datos MySQL (13 tablas) • Stored Procedures y Triggers Responsabilidad: Persistencia de datos, transacciones ACID, integridad referencial

3. COMPONENTES DEL SERVIDOR

3.1 Servicios Web SOAP

El servidor expone tres servicios web SOAP que encapsulan toda la lógica de negocio del sistema:

3.1.1 ServicioAutenticacion

Namespace: ec.edu.monster.ws.ServicioAutenticacion

Operaciones expuestas:

- **login(usuario, clave):** Autentica un empleado y retorna sus datos
- **registrarEmpleado(empleado):** Registra un nuevo empleado en el sistema
- **cambiarClave(codigo, claveActual, claveNueva):** Actualiza la contraseña de un empleado

3.1.2 ServicioTransaccion

Namespace: ec.edu.monster.ws.ServicioTransaccion

Operaciones expuestas:

- **realizarDeposito(datos):** Incrementa el saldo de una cuenta
- **realizarRetiro(datos):** Decrementa saldo aplicando ITF y costo por movimiento
- **realizarTransferencia(datos):** Transfiere fondos entre dos cuentas de forma atómica

3.1.3 ServicioReporte

Namespace: ec.edu.monster.ws.ServicioReporte

Operaciones expuestas:

- **obtenerMovimientos(codigoCuenta):** Retorna todos los movimientos de una cuenta
- **obtenerMovimientosPorRango(codigoCuenta, fechaInicio, fechaFin):** Retorna movimientos filtrados por fechas

3.2 Capa de Servicios

La capa de servicios implementa la lógica de negocio del sistema y es consumida por los servicios web SOAP. Cada servicio encapsula operaciones relacionadas y mantiene la separación de responsabilidades.

Servicio	Responsabilidades
AutenticacionService	Validación de credenciales, gestión de empleados, cambio de contraseñas, generación de códigos
TransaccionService	Procesamiento de depósitos, retiros y transferencias. Cálculo de ITF y costos. Validación de saldos. Gestión de transacciones ACID.
ReporteService	Consulta de movimientos históricos, filtrado por fechas, generación de reportes detallados

3.3 Capa de Acceso a Datos (DAO)

El patrón DAO (Data Access Object) abstrae y encapsula todo el acceso a la base de datos. Cada DAO es responsable de las operaciones CRUD sobre una entidad específica.

DAO	Operaciones Principales
EmpleadoDAO	validarLogin(), registrar(), cambiarClave(), buscarPorUsuario(), generarCodigo()
CuentaDAO	buscarPorCodigo(), actualizarSaldo(), validarClave(), obtenerSaldo(), cambiarEstado()
MovimientoDAO	registrarMovimiento(), obtenerMovimientos(), obtenerPorRango(), obtenerUltimoNumero()
ClienteDAO	buscarPorCodigo(), registrar(), actualizar(), listarTodos()

3.4 Objetos de Transferencia (DTO)

Los DTOs son objetos especializados para transferir datos entre capas, especialmente entre el servidor y los clientes. Son serializables y optimizados para transmisión por SOAP.

- **RespuestaDTO:** Encapsula respuestas genéricas (éxito/error, mensaje, datos)
- **TransaccionDTO:** Datos para depósitos y retiros (cuenta, importe, empleado, clave)
- **TransferenciaDTO:** Datos para transferencias (cuenta origen, destino, importe)
- **MovimientoDetalleDTO:** Información completa de un movimiento para reportes
- **DepositoResultDTO:** Resultado de un depósito con saldos anterior y actual

4. PATRONES DE DISEÑO

4.1 Patrón DAO (Data Access Object)

Propósito: Abstraer y encapsular el acceso a la base de datos

Beneficios:

- Separación entre lógica de negocio y acceso a datos
- Facilita el cambio de proveedor de base de datos
- Centraliza las consultas SQL en una sola ubicación
- Facilita las pruebas unitarias con DAOs simulados

4.2 Patrón DTO (Data Transfer Object)

Propósito: Transferir datos entre capas minimizando el número de llamadas

Beneficios:

- Reduce el acoplamiento entre capas
- Optimiza la transmisión de datos por red
- Permite versionar estructuras de datos
- Evita exponer entidades de dominio directamente

4.3 Patrón Service Layer

Propósito: Encapsular la lógica de negocio en una capa de servicios

Beneficios:

- Centraliza las reglas de negocio
- Facilita la reutilización de lógica de negocio
- Permite transacciones coordinadas entre múltiples DAOs
- Mejora la mantenibilidad del código

4.4 Patrón Singleton (Gestor de Conexiones)

Propósito: Garantizar una única instancia del gestor de conexiones a la base de datos

Implementación: Clase ConexionDB con configuración centralizada

Beneficios:

- Centraliza la configuración de la conexión
- Gestiona el pool de conexiones eficientemente
- Facilita el cambio de credenciales o servidor

5. INFRAESTRUCTURA Y DESPLIEGUE

5.1 Base de Datos en Railway

La base de datos MySQL está desplegada en Railway, una plataforma de cloud computing moderna que proporciona:

- **Alta disponibilidad:** Infraestructura redundante con respaldos automáticos
- **Escalabilidad automática:** Ajuste de recursos según la demanda
- **Acceso seguro:** Conexiones cifradas SSL/TLS
- **Monitoreo:** Métricas de rendimiento y uso en tiempo real
- **Respaldos automáticos:** Snapshots diarios con retención de 7 días

Configuración de Conexión:

Host: mysql.railway.internal

Puerto: 3306

Base de datos: eurekabank

Usuario: eureka

5.2 Servidor de Aplicaciones

El servidor SOAP puede desplegarse en cualquier servidor de aplicaciones compatible con Jakarta EE:

- **Apache Tomcat** (recomendado para desarrollo)
- **WildFly** (para producción enterprise)
- **GlassFish** (servidor de referencia Jakarta EE)
- **Payara** (fork optimizado de GlassFish)

5.3 Requisitos del Sistema

Componente	Requisitos
Servidor	JDK 17+, 2GB RAM mínimo, 4GB recomendado, 1GB espacio en disco, Conexión a internet estable
Clientes	.NET Framework 4.8+ o .NET 6+, Windows 10/11, 512MB RAM, Navegador web moderno (para cliente web)

6. SEGURIDAD

6.1 Autenticación

El sistema implementa autenticación basada en credenciales (usuario y contraseña) con las siguientes características:

- Validación de credenciales contra base de datos
- Recomendación: Implementar hashing de contraseñas (bcrypt, PBKDF2)